

JAVA 17

Panoramica

Java 17 è una release **LTS (Long-Term Support)**, cioè una versione con supporto a lungo termine. Le precedenti LTS erano Java 8 e Java 11. Le novità principali introdotte in Java 17 rispetto a Java 11 sono cinque: **classi sigillate (sealed classes)**, **record**, **espressioni switch**, **pattern matching e blocchi di testo (text blocks)**. Sono esattamente i cinque argomenti trattati nelle lezioni successive.

Sealed Classes and Interfaces (classi e interfacce sigillate)

Definizione

Il problema che le classi sigillate risolvono nasce da due estremi dell'ereditarietà in Java:

- **Ereditarietà normale**: qualsiasi classe può ereditare da qualsiasi altra classe. Massima apertura.
- **Classe final**: impedisce a *chiunque* di ereditare da quella classe. Massima chiusura — la classe non può mai diventare un supertipo.

Una **classe sigillata** è la **via di mezzo**: permette di controllare *l'ambito dell'ereditarietà*, specificando esattamente *quali* classi possono ereditare da essa. Non "tutte" e non "nessuna", ma "solo queste che indico io".

Lo stesso meccanismo funziona anche con le **interfacce**: puoi dichiarare quali classi sono autorizzate a implementarle.

Le due parole chiave

- `sealed` → dichiara la classe/interfaccia come sigillata.
- `permits` → elenca le sottoclassi/implementazioni consentite.

Regola fondamentale della gerarchia

Quando una classe entra in una gerarchia sigillata, **ogni** sottoclasse deve dichiarare esplicitamente uno di questi tre stati:

Modificatore	Significato
<code>final</code>	Nessun'altra classe può estenderla → chiude la gerarchia
<code>sealed</code>	Continua a controllare i propri sottotipi con <code>permits</code>
<code>non-sealed</code>	Riapre l'ereditarietà → qualsiasi classe può estenderla

Come dice la trascrizione: `final` è l'estremo "chiuso", `non-sealed` è l'estremo "aperto", e `sealed` è la via di mezzo.

Esempio: gerarchia di veicoli

```
// Classe sigillata: solo Car può estenderla
```

```

public sealed class Vehicle permits Car {
}

// Car è a sua volta sigillata: solo Ford e Volvo possono estenderla
public sealed class Car extends Vehicle permits Ford, Volvo {
}

// Ford e Volvo chiudono la gerarchia
public final class Ford extends Car {
}

public final class Volvo extends Car {
}

```

Cosa impedisce questa struttura? Se provassi a scrivere:

```

public final class Table extends Vehicle { } // ERRORE del compilatore

```

ottiene un errore di compilazione, perché `Vehicle` permette **solo** `Car` come sottotipo. Una tabella non è un veicolo, e il compilatore lo garantisce a monte.

La gerarchia finale è: `Vehicle` → ha un solo sottotipo `Car`; `Car` → ha due sottotipi `Ford` e `Volvo`; entrambi `final`, quindi la gerarchia è completamente chiusa e definita.

Esempio con interfaccia sigillata

```

public sealed interface Drivable permits Vehicle {
}

public sealed class Vehicle implements Drivable permits Car {
}

```

Qui `Drivable` dichiara che **l'unica** classe autorizzata a implementarla è `Vehicle`. Se scrivessi `class Table implements Drivable`, sarebbe di nuovo un **errore del compilatore**.

Il vantaggio in sintesi

Con classi e interfacce sigillate specifichi con precisione l'ambito della gerarchia di ereditarietà. Qualsiasi uso di una classe non consentita (con `extends` o `implements`) viene bloccato **a tempo di compilazione**, non a runtime.

Records (record)

Definizione

Un **record** è un tipo speciale di classe pensato per **evitare il codice boilerplate** (il codice ripetitivo e "meccanico" che scriveresti a mano). I record sono definiti come "**data carriers**", cioè contenitori di dati. In pratica ti risparmiano moltissima digitazione.

Proprietà fondamentali dei record

- **Immutabili**: una volta creato l'oggetto, non puoi più modificarlo. Se tenti di modificarlo, in background viene creato un *nuovo* oggetto e l'originale resta intatto (e diventa candidato per il garbage collector).
- **final per impostazione predefinita**: non puoi creare sottotipi di un record.
- **Non puoi estendere un record da un'altra classe**: i record estendono già implicitamente la classe `Record`, e Java non supporta l'ereditarietà multipla (è lo stesso meccanismo delle `enum`, che estendono implicitamente `Enum`).
- **Possono avere** campi statici, metodi statici e metodi di istanza.
- **NON possono avere campi di istanza aggiuntivi**: gli unici campi di istanza sono i **componenti** dichiarati tra le parentesi tonde. Non puoi aggiungerne altri dentro il corpo.
- **Possono implementare interfacce**.

Cosa genera automaticamente un record

Quando dichiarare un record, il compilatore genera per te, in background:

1. **Campi di istanza** `private final` — uno per ogni componente.
2. **Costruttore canonico** — prende come argomenti tutti i componenti e inizializza i campi.
3. **`toString()`** — rappresentazione in stringa con nomi e valori dei componenti.
4. **`equals()` e `hashCode()`** — due record sono uguali se sono dello stesso tipo e hanno tutti i componenti uguali.
5. **Metodi accessori pubblici con lo stesso nome del componente** (attenzione: *non* `getRegNo()`, ma semplicemente `regNo()`).

Il confronto: classe normale vs record

Una classe `Car` scritta a mano — con due campi `final`, costruttore, due getter, e `toString/equals/hashCode` — occupa **oltre 40 righe di codice**:

```
public class Car {  
    private final String regNo;  
    private final String owner;  
  
    public Car(String regNo, String owner) {  
        this.regNo = regNo;  
        this.owner = owner;  
    }  
}
```

```

public String getRegNo() { return regNo; }
public String getOwner() { return owner; }

@Override public String toString() { /* ... */ }
@Override public boolean equals(Object o) { /* ... */ }
@Override public int hashCode() { /* ... */ }
}

```

Il record equivalente è **una sola riga**:

```

public record CarRecord(String regNo, String owner) { }

```

Questa singola riga crea i due campi `final`, il costruttore canonico, i metodi accessori `regNo()` e `owner()`, e sovrascrive `toString`, `equals` e `hashCode`. In background è quasi identico alla classe `Car` completa.

Come si usano (differenza nei metodi accessori)

// Classe normale

```

Car car = new Car("231G4321", "Mary Bloggs");
System.out.println(car);           // chiama toString()
System.out.println(car.getOwner()); // getter tradizionale
System.out.println(car.getRegNo());

```

// Record

```

CarRecord cr = new CarRecord("231G4321", "Mary Bloggs");
System.out.println(cr);           // chiama toString()
System.out.println(cr.owner());   // NON getOwner(), ma owner()
System.out.println(cr.regNo());   // NON getRegNo(), ma regNo()

```

Il risultato funzionale è identico; cambia solo il nome dei metodi accessori.

Campo di istanza non consentito

```

public record CarRecord(String regNo, String owner) {
    // private String age; // ERRORE: "instance field is not allowed in a record"
}

```

Se ti serve un campo `age`, devi aggiungerlo come **componente** nella dichiarazione: `public record CarRecord(String regNo, String owner, int age)`.

Metodi statici, campi statici e metodi di istanza

```
public record CarRecord(String regNo, String owner) {

    // Metodo di istanza

    public boolean isNewCar() {
        // prende i primi due caratteri della targa e li confronta con l'anno corrente
        return regNo.substring(0, 2).equals(String.valueOf(currentYear));
    }

    // Campo statico

    static int currentYear = 23;

    // Metodo statico che restituisce un CarRecord vuoto

    public static CarRecord createBlankCarRecord() {
        return new CarRecord("  ", "");
    }
}
```

Costruttore canonico personalizzato (validazione dei dati)

Puoi sovrascrivere il costruttore canonico, tipicamente per **validare** i dati:

```
public record CarRecord(String regNo, String owner) {

    // Costruttore canonico personalizzato

    public CarRecord(String regNo, String owner) {
        if (regNo.length() <= 4) {
            throw new IllegalArgumentException("regNo troppo corto");
        }

        this.regNo = regNo;
        this.owner = owner;
    }
}
```

Qui prendi il controllo di parte del comportamento predefinito: la parte personalizzata è l'`if` di validazione, il resto (le assegnazioni `this.regNo = regNo`) è ciò che sarebbe stato generato comunque.

Nota: nell'esempio della trascrizione, `createBlankCarRecord()` con una targa vuota lancia l'`IllegalArgumentException` proprio perché la lunghezza era insufficiente.

Costruttore compatto (compact constructor)

È una **variante concisa** del costruttore canonico, **specifica dei record** (non puoi usarla in una classe normale). Tolgi i parametri tra parentesi tonde e le assegnazioni finali, perché il compilatore le ricava da solo dai componenti:

```
public record CarRecord(String regNo, String owner) {  
    // Costruttore compatto: niente parentesi tonde, niente assegnazioni esplicite  
    public CarRecord {  
        if (regNo.length() <= 4) {  
            throw new IllegalArgumentException("regNo troppo corto");  
        }  
        // this.regNo = regNo; e this.owner = owner; sono impliciti  
    }  
}
```

Il compilatore sintetizza sia la lista dei parametri sia l'inizializzazione finale. Devi scrivere solo la logica personalizzata (`if`).

Costruttori non canonici

Se aggiungi un costruttore **diverso** da quello canonico (per esempio uno senza argomenti), devi rispettare una regola: **"un costruttore record non canonico deve delegare a un altro costruttore"**. In pratica deve, prima o poi, chiamare il costruttore compatto/canonico:

```
public record CarRecord(String regNo, String owner) {  
    public CarRecord { // costruttore compatto  
        if (regNo.length() <= 4)  
            throw new IllegalArgumentException();  
    }  
  
    // Costruttore non canonico: DEVE delegare  
    public CarRecord() {  
        this(" ", "sconosciuto"); // delega al canonico  
    }  
}
```

Se ometti la delega (`this(...)`), ottieni l'errore *"non-canonical constructor must delegate to another constructor"*.

Sovrascrivere un accessore

Puoi anche sovrascrivere un singolo metodo accessore (es. `owner()`) se vuoi modificarne il comportamento di default. *(La trascrizione si interrompe qui, mentre stava iniziando questo esempio.)*

Switch Expressions and Pattern Matching

Contesto storico

- Le **espressioni switch** sono state introdotte come *funzionalità di anteprima* (preview) in **Java 12** e sono diventate permanenti in **Java 14**.
- L'istruzione **yield** è stata introdotta in **Java 13** per supportarle.
- Il **pattern matching nello switch** è invece una **funzionalità di anteprima in Java 17** (potrebbe cambiare o sparire nelle versioni future).

Cos'è una funzionalità di anteprima (preview feature)? È una funzione già progettata e implementata ma **non ancora permanente**. Serve a far testare la novità a un vasto pubblico di sviluppatori prima che venga resa definitiva. Le preview sono **disattivate per impostazione predefinita** e vanno abilitate esplicitamente (es. nelle impostazioni del progetto in IntelliJ, impostando il language level a "17 Preview", e aggiungendo `--enable-preview` alle configurazioni di esecuzione).

Lo switch tradizionale e i suoi problemi

```
enum Direction { NORTH, SOUTH, EAST, WEST }
```

```
Direction dx = Direction.NORTH;
```

```
int numLetters = 0;
```

```
switch (dx) {  
    case NORTH:  
    case SOUTH:  
        numLetters = 5;  
        break;           // <-- obbligatorio  
    case EAST:  
    case WEST:  
        numLetters = 4;  
        break;  
}
```

Due problemi evidenti:

- Logica di "fall through"**: devi ricordarti di inserire il `break` in ogni caso, altrimenti l'esecuzione "cade" nel caso successivo. È noioso e fonte di bug.
- Ridondanza**: la variabile `numLetters` viene inizializzata in più punti. Con uno switch grande (es. i 12 mesi dell'anno) il problema si amplifica.

L'espressione switch (sintassi a freccia)

Concetto chiave: **un'espressione produce un singolo valore**. Un'espressione switch non fa eccezione — restituisce un valore che puoi assegnare o passare direttamente.

```
int numLetters = switch (dx) {  
    case NORTH, SOUTH -> 5;  
    case EAST, WEST  -> 4;  
    default          -> 0;  
};
```

Vantaggi:

- **Niente break**: la sintassi a freccia (->) non ha fall-through.
- **Più concisa**: casi multipli separati da virgola su una riga.
- **Restituisce un valore** direttamente.
- **Copertura obbligatoria**: le etichette devono coprire *tutti* i valori possibili. In genere questo richiede un default alla fine. Se lo ometti (quando non tutti i casi sono coperti), ottieni un **errore di compilazione**.

L'istruzione yield (sintassi a due punti)

Se preferisci la sintassi tradizionale con i **due punti** (:), per restituire un valore da un'espressione switch devi usare yield:

```
int numLetters = switch (dx) {  
    case NORTH, SOUTH:  
        yield 5;  
    default:  
        yield 4;  
};
```

Regola: con la **freccia** (->) il valore è restituito implicitamente; con i **due punti** (:) devi usare yield esplicitamente per restituire il valore.

Pattern matching nello switch

Permette di ramificare in base al **tipo** di un oggetto, ottenendo automaticamente una variabile già "castata" del tipo giusto.

```
static String whatType(Object o) {  
    return switch (o) {  
        case String s -> "È una stringa";  
        case Integer i -> "È un intero";  
        case null     -> "È null";  
        default       -> "Non riconosciuto";  
    };  
}
```

Quando o è una String, Java fa il match e ti fornisce direttamente la variabile s di tipo String con cui lavorare (senza cast manuale). Anche qui la copertura deve essere completa: senza default ottieni un errore.

Esempio d'uso: passando "ABC", 1, 2, null e 32.39 (che viene "impacchettato" come Double), l'output è Stringa, Intero, Intero, null, Non riconosciuto (il double non è coperto).

Pattern guarded (pattern protetti)

Puoi **interrogare** la variabile fornita dal match, aggiungendo una condizione:

```
static String typeInfo(Object o) {  
    return switch (o) {  
        case String s when s.startsWith("A") -> s + " (inizia con A)";  
        case Integer i when i > 10          -> i + " (maggiore di dieci)";  
        default                             -> "Non riconosciuto";  
    };  
}
```

Esempi di comportamento:

- "ABC" → match sul primo caso (inizia con "A" maiuscola).
- "abc" → **non** corrisponde (il match è case-sensitive) → default "Non riconosciuto".
- 12 → match sul secondo caso (> 10).
- 8 → fallisce il test > 10 → default.

Questo è utile quando vuoi flussi di controllo diversi per sottotipi di un supertipo comune.

Esempio completo con classi sigillate + pattern matching

Qui si combinano gli argomenti precedenti: una gerarchia sigillata usata dentro il pattern matching.

```
sealed abstract class Vehicle permits Car, Bought, Train { }  
  
final class Car extends Vehicle {  
    public void onRoad() { // metodo personalizzato, solo di Car  
        System.out.println("Posso muovermi sulla strada");  
    }  
}  
  
final class Bought extends Vehicle { }  
final class Train extends Vehicle { }
```

Versione con if e pattern matching (disponibile da Java 16):

```
static void whatType(Vehicle v) {  
    if (v instanceof Car car) { // match + variabile 'car' pronta  
        System.out.println("È una macchina");  
        car.onRoad(); // chiamo il metodo personalizzato  
    } else if (v instanceof Bought b) {  
        System.out.println("È una barca");  
    } else if (v instanceof Train t) {  
        System.out.println("È un treno");  
    } else {  
        throw new IllegalArgumentException();  
    }  
}
```

```
}
```

Come spiega Oracle (citato nella lezione), il pattern matching consente di **estrarre condizionatamente i dati dagli oggetti con codice più conciso e robusto**. Il `instanceof Car car` fa due cose in una: verifica il tipo e, se coincide, ti fornisce già il riferimento `car` tipizzato.

Versione con espressione switch e pattern matching:

```
static void whatTypeSwitch(Vehicle v) {  
    switch (v) {  
        case Car c -> c.onRoad();           // variabile c disponibile  
        case Bought b -> System.out.println("È una barca");  
        case Train t -> System.out.println("È un treno");  
    }  
}
```

Il bello: in ogni ramo hai una **variabile** già pronta per chiamare i metodi specifici di quel sottotipo. **Avvertenza importante:** il pattern matching nello switch è solo una *preview* in Java 17 e può essere rimosso o modificato **senza preavviso** nelle versioni future. È utile però osservare la direzione che il linguaggio sta prendendo.

Text Blocks (blocchi di testo)

Definizione

Un **blocco di testo** è un **oggetto String** a tutti gli effetti. Di conseguenza condivide tutte le proprietà delle stringhe: è **immutabile** ed è **internato** ("interned", cioè condiviso nel pool di stringhe). Puoi chiamare su di esso qualsiasi metodo di `String`.

Serve a scrivere comodamente testo su **più righe**, rendendo il codice molto più leggibile.

Sintassi

- Un blocco di testo inizia con il **delimitatore di apertura: tre doppi apici ("")** seguiti **obbligatoriamente da una nuova riga**.
- Il testo **non può** stare sulla stessa riga del delimitatore di apertura.
- Termina con il **delimitatore di chiusura** ("").

```
String name = ""  
    Sean Kennedy"";
```

Il delimitatore di apertura è seguito da un ritorno a capo; il testo "Sean Kennedy" sta su una riga separata; poi chiude con "". Questo è un blocco valido.

Blocchi validi e non validi

```
String tb1 = ""abc"";    // ERRORE: tutto su una riga
```

```
String tb2 = ""abc  
    "";    // ERRORE: testo sulla stessa riga del delimitatore di apertura
```

```
String tb3 = ""
```

```
abc"""; // CORRETTO
```

La regola: serve sempre una nuova riga subito dopo "" di apertura.

Un blocco di testo È una stringa (dimostrazione)

```
String name = "Sean Kennedy";

String nameTB = ""

    Sean Kennedy"";

name.equals(nameTB); // true -> stesso contenuto

name == nameTB;     // true -> sono internati (stesso riferimento nel pool)

nameTB.substring(5); // "Kennedy" -> i metodi di String funzionano
```

Virgolette incorporate: niente escape

Nelle stringhe tradizionali devi fare l'**escape** delle virgolette doppie interne. Nei blocchi di testo **no**, il che li rende molto più leggibili:

```
// Stringa tradizionale: virgolette da "escapare"

String quote = "Nothing is good or bad but thinking makes it so, \"said\" Hamlet";

// Blocco di testo: virgolette incorporate senza escape

String quoteTB = ""

    Nothing is good or bad but thinking makes it so, "said" Hamlet"";
```

Il vantaggio è enorme con testi come il **JSON**, dove la versione tradizionale è piena di \n e \" (difficile da leggere), mentre la versione a blocco di testo è pulita e corrisponde esattamente all'output:

```
// Tradizionale: illeggibile

String json = \"{\\n \\\"nome\\\": \\\"Sean\\\",\\n \\\"eta\\\": 30\\n}\";
```

// Blocco di testo: leggibile, identico in output

```
String jsonTB = ""
```

```
{  
  
  "nome": "Sean",  
  
  "eta": 30  
  
}"";
```

Nuova riga finale e posizione del delimitatore di chiusura

Le tre virgolette di chiusura determinano se c'è o meno una **nuova riga finale**:

// Delimitatore di chiusura su riga separata -> C'È una nuova riga alla fine

```
String book1 = ""
```

```
  Java  
  
  Memory  
  
  Management  
  
""; // equivale a "Java\nMemory\nManagement\n"
```

// Delimitatore subito dopo l'ultimo carattere -> NIENTE nuova riga finale

```
String book2 = ""
```

```
  Java  
  
  Memory  
  
  Management""; // equivale a "Java\nMemory\nManagement"
```

Spazi "incidentali" e indentazione

Questa è la parte più sottile. La **spaziatura a sinistra** è determinata dalla posizione del **delimitatore di chiusura**, oppure dal **primo carattere non-spazio** che si incontra — a seconda di quale viene per primo (il più a sinistra).

- Tutti gli spazi a sinistra di quella posizione sono considerati **incidentali** e vengono **rimossi** da tutte le righe.
- Gli spazi a destra di quella posizione sono considerati **obbligatorî** e vengono **mantenuti** (servono per l'indentazione relativa del contenuto).

Nota: questo algoritmo di rimozione degli spazi incidentali funziona solo se il delimitatore di chiusura è **su una riga a sé stante**.

Esempio pratico con JSON: se sposti il delimitatore di chiusura più a sinistra o più a destra, cambi la "posizione di partenza" da cui si misura l'indentazione:

```
String json = ""
```

```
{  
    "nome": "Sean"  
}  
;
```

```
// ^ il delimitatore di chiusura qui stabilisce il margine sinistro
```

Se il delimitatore di chiusura è tutto a sinistra, le parentesi graffe finiscono a sinistra e i campi interni ("nome") restano indentati **relativamente** di due spazi. Se sposti il delimitatore a destra di due spazi, tutto il blocco si sposta di conseguenza, mantenendo l'indentazione relativa interna. Quando invece il primo carattere non-spazio (es. la {) è più a sinistra del delimitatore, è quello a stabilire il livello incidentale, e tutti gli spazi alla sua sinistra vengono rimossi da ogni riga.

In sintesi sui blocchi di testo

I blocchi di testo sono uguali alle stringhe (immutabili, internati, con tutti i metodi di `String`), facilitano moltissimo la scrittura di testo su più righe e rendono il codice più leggibile. Bisogna però essere consapevoli del **significato implicito della posizione del delimitatore di chiusura** (nuova riga finale + gestione degli spazi incidentali) e ricordare che si inizia sempre con `""` di apertura **seguito da una nuova riga**.